

INSTITUTO INFNET
ENGENHARIA DA COMPUTACAO

RELATORIO TECNICO
TP4 - SISTEMAS DIGITAIS EMBARCADOS

Modulos aritmeticos, rotinas numericas avancadas e comunicacao ARM-FPGA



Aluno: João Paulo Fonseca de Araújo
E-mail: joao.paraujo@al.infnet.edu.br
Projeto de Bloco: Sistemas Digitais Embarcados
[26E2_5]

Rio de Janeiro
2026

1 INTRODUCAO

Este relatorio documenta o desenvolvimento do TP4, que da continuidade ao projeto integrado em Sistemas Digitais Embarcados iniciado no TP3. O trabalho amplia a complexidade logica e aritmetica do protótipo em duas frentes principais: no FPGA, a construcao de uma unidade aritmetica multiplicador-acumulador (MAC) utilizando recursos internos de memoria (BRAM) e processamento dedicado (DSP); e, no ARM, o desenvolvimento de rotinas numericas avancadas, incluindo aritmetica multi-palavra, conversao entre inteiro e ponto flutuante, tabelas de consulta, manipulacao bitwise e paralelizacao vetorial via NEON SIMD. Adicionalmente, foi implementado um mecanismo de comunicacao simulada entre ARM e FPGA com deteccao de erros via checksum e registro de telemetria.

A mesma restricao de ambiente do TP3 permanece valida: nao ha disponibilidade do hardware Raspberry Pi Zero 2W fisico. Por esse motivo, toda a frente de Assembly ARM64 (rotinas numericas, NEON SIMD e comunicacao simulada) foi desenvolvida e validada exclusivamente por emulacao com QEMU (qemu-aarch64), com depuracao via GDB remoto e analise estatica via objdump. A frente de FPGA foi integralmente sintetizada e validada na placa Tang Nano 4K fisica, com evidencias reais de funcionamento, incluindo confirmacao do uso efetivo de blocos BRAM e DSP no relatorio de sintese do Gowin EDA.

2 MODULOS ARITMETICOS E ESTRUTURAS DE ARMAZENAMENTO NO FPGA

2.1 Descricao da unidade aritmetica projetada

Foi projetada uma unidade multiplicador-acumulador (MAC) com saturacao aritmetica de 32 bits, no estilo de um acumulador FIR simplificado. A cada acionamento, o sistema le um coeficiente e uma amostra armazenados em memorias de bloco (BRAM), multiplica os dois valores utilizando um multiplicador dedicado (DSP) e acumula o resultado, protegendo contra overflow atraves de saturacao no valor maximo ou minimo representavel em 32 bits. O modulo fsm_control ja validado fisicamente no TP3 foi mantido no projeto integrado.

2.2 Arquitetura hierarquica e uso de BRAM/DSP

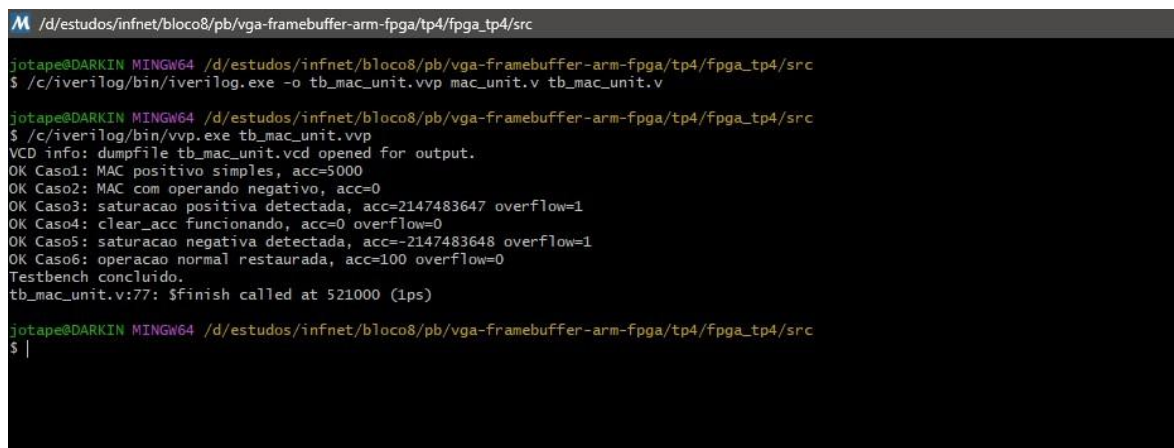
O projeto foi organizado em seis modulos Verilog: coeff_rom.v e sample_rom.v (memorias de 64 posicoes de 16 bits, com leitura sincrona registrada e o atributo de sintese ram_style="block" para forcar o mapeamento em blocos BSRAM dedicados, em vez de logica distribuida), mac_unit.v (multiplicador de 16x16 bits e acumulador saturado de 32 bits), mac_control.v (maquina de estados de tres fases que orquestra o acesso

sequencial a memoria, respeitando a latencia de um ciclo de clock da leitura sincrona) e top_mac.v (modulo de topo que integra os blocos aos botoes e ao LED onboard da Tang Nano 4K).

A largura de 16x16 bits para o multiplicador foi escolhida de forma deliberada por ser compativel com os multiplicadores de 18x18 bits disponiveis nos blocos DSP do dispositivo GW1NSR-4C, favorecendo a inferencia automatica de hardware dedicado pelo sintetizador Gowin, em vez de logica generica implementada em LUTs.

2.3 Testbench e simulacao

O testbench tb_mac_unit.v cobre seis casos de teste sobre o modulo aritmetico isoladamente: multiplicacao-acumulacao positiva simples, operando negativo, saturacao no limite maximo positivo (32 bits), reinicializacao via clear_acc, saturacao no limite minimo negativo e retomada de operacao normal apos a saturacao. Todos os seis casos foram validados com sucesso.



```

M /d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp4/fpga_tp4/src
jotape@DARKIN MINGW64 /d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp4/fpga_tp4/src
$ /c/iverilog/bin/iverilog.exe -o tb_mac_unit.vvp mac_unit.v tb_mac_unit.v
jotape@DARKIN MINGW64 /d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp4/fpga_tp4/src
$ /c/iverilog/bin/vvp.exe tb_mac_unit.vvp
VCD info: dumpfile tb_mac_unit.vcd opened for output.
OK Caso1: MAC positivo simples, acc=5000
OK Caso2: MAC com operando negativo, acc=0
OK Caso3: saturacao positiva detectada, acc=2147483647 overflow=1
OK Caso4: clear_acc funcionando, acc=0 overflow=0
OK Caso5: saturacao negativa detectada, acc=-2147483648 overflow=1
OK Caso6: operacao normal restaurada, acc=100 overflow=0
Testbench concluido.
tb_mac_unit.v:77: $finish called at 521000 (1ps)
jotape@DARKIN MINGW64 /d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp4/fpga_tp4/src
$ |

```

Figura 1 - Execucao do testbench tb_mac_unit.v via Icarus Verilog, confirmando os seis casos de teste da unidade aritmetica, incluindo deteccao de saturacao positiva e negativa.

Um segundo testbench de integracao, tb_top_mac.v, valida o sistema completo incluindo o acesso as memorias BRAM e o controlador de estados, simulando oito passos consecutivos de operacao e confirmando a evolucao do endereco, a leitura das memorias e a acumulacao ate a ocorrencia de saturacao.

```

M /d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp4/fpga_tp4/src
jotape@DARKIN MINGW64 /d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp4/fpga_tp4/src
$ /c/iverilog/bin/iverilog.exe -o tb_top_mac.vvp coeff_rom.v sample_rom.v mac_unit.v mac_control.v tb_top_mac.v
jotape@DARKIN MINGW64 /d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp4/fpga_tp4/src
$ /c/iverilog/bin/vvp.exe tb_top_mac.vvp
VCD info: dumpFile tb_top_mac.vcd opened for output.
Apos 8 passos: addr=8 acc=1056880000 overflow=0
Testbench de integracao concluido.
tb_top_mac.v:56: $finish called at 721000 (1ps)

jotape@DARKIN MINGW64 /d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tp4/fpga_tp4/src
$ |

```

Figura 2 - Testbench de integracao tb_top_mac.v, executando oito passos consecutivos de leitura das BRAM e acumulacao, confirmando o comportamento esperado do sistema completo.

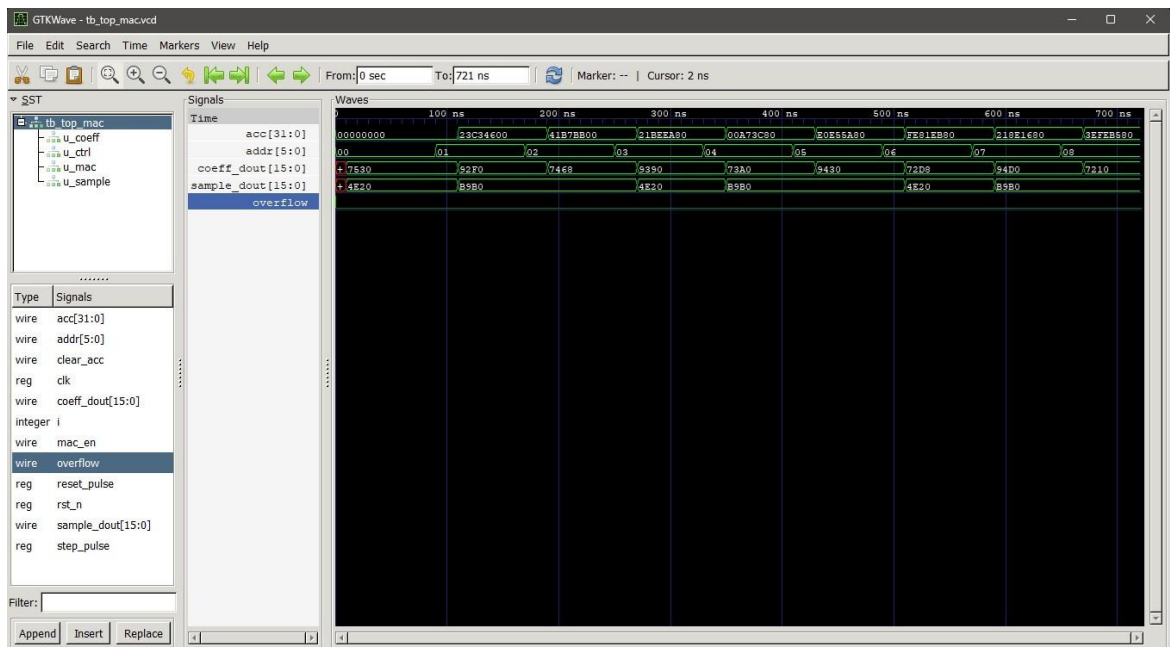
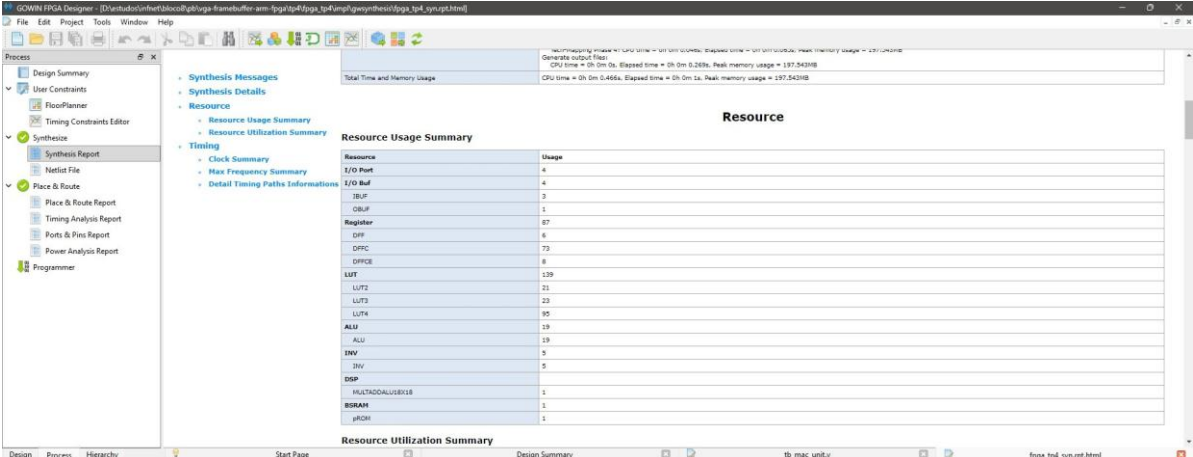


Figura 3 - Forma de onda da simulacao de integracao no GTKWave, evidenciando a evolucao do endereco, os valores lidos das memorias coeff_dout/sample_dout e o acumulador ao longo do tempo, com atencao a latencia de leitura sincrona.

2.4 Sintese, confirmacao de uso de BRAM e DSP

O projeto foi sintetizado no Gowin EDA para o dispositivo GW1NSR-LV4CQN48PC6/I5. O relatorio de sintese confirma explicitamente a inferencia de um bloco DSP (recurso MULTADDALU18X18, quantidade igual a 1) e de um bloco de memoria BSRAM (recurso pROM, quantidade igual a 1), validando que as decisoes de projeto (largura do multiplicador compativel com o DSP, atributo ram_style nas memorias) surtiram o efeito pretendido durante a sintese.



Resource	Usage
EIO Port	4
EIO Buf	4
IOBUF	3
IOBUF	1
Register	87
DFF	6
DFPC	73
DFPC	6
LUT	139
LUT2	23
LUT3	23
LUT4	95
ALU	19
ALU	19
SRV	5
SRV	5
DSP	1
MULTADDALU18X18	1
BSRAM	1
pROM	1

Figura 4 - Relatório de síntese do Gowin EDA (Resource Usage Summary), confirmando a utilização de um bloco DSP (MULTADDALU18X18) e um bloco BSRAM (pROM) no projeto.

2.5 Validação física na Tang Nano 4K

Após a síntese e o Place and Route, o bitstream foi gravado fisicamente na placa Tang Nano 4K via Gowin Programmer. O comportamento físico observado confirma o funcionamento do sistema completo: ao pressionar o botão onboard USR_KEY_1 repetidamente, o LED emite um flash curto de confirmação a cada passo executado com sucesso; após o oitavo acionamento, o acumulador atinge o limite de saturação de 32 bits, e o LED passa a piscar continuamente em alta frequência, indicando o estado de overflow detectado pelo hardware. O botão USR_KEY_2 reinicializa o acumulador e o endereço, retornando o LED ao comportamento normal.

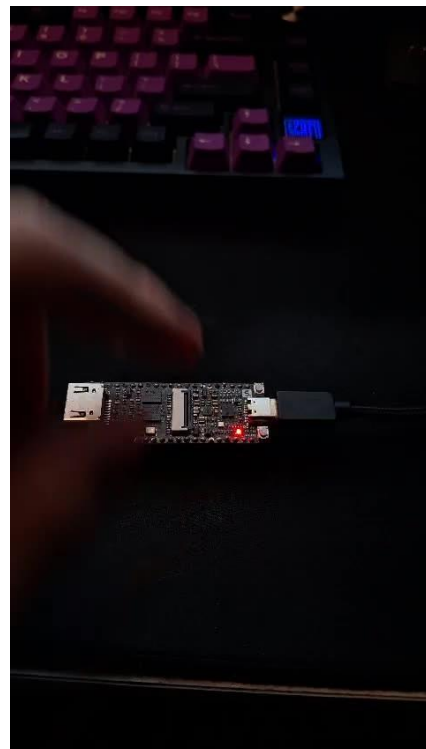
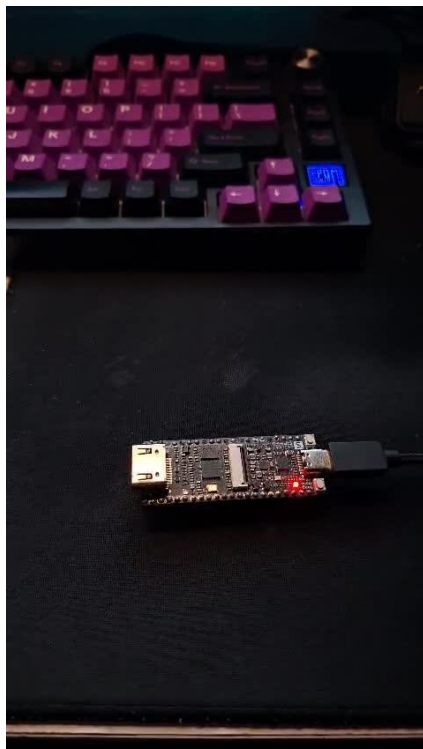


Figura 5 - Validacao fisica na Tang Nano 4K: LED aceso indicando resposta ao acionamento do botao onboard USR_KEY_1 (esquerda) e interacao direta com o botao fisico durante o teste de saturacao (direita). Quadros extraídos do video de demonstracao do funcionamento do modulo MAC.

2.6 Requisitos de avaliacao atendidos nesta etapa

Implementacao de unidade aritmetica em Verilog (somador/multiplicador/acumulador): atendido, ver Figuras 1 e 2.

Uso de BRAM para armazenamento de tabelas: atendido (coeff_rom e sample_rom), confirmado no relatorio de sintese, ver Figura 4.

Uso de DSP para otimizacao de calculos: atendido (multiplicador 16x16 mapeado para MULTADDALU18X18), ver Figura 4.

Waveforms com atencao a quantizacao e comportamento numerico: atendido, ver Figura 3.

Sintese e teste fisico na Tang Nano 4K com evidencias fotograficas: atendido, ver Figura 5.

3 ROTINAS DE MANIPULACAO NUMERICA AVANCADA EM ASSEMBLY ARM 64 BITS

Restricao de ambiente: nao ha Raspberry Pi Zero 2W fisico disponivel. Todas as rotinas descritas nesta secao foram desenvolvidas e validadas por emulacao via QEMU (qemu-aarch64), com depuracao remota via gdb-multiarch.

3.1 Aritmetica multi-palavra

A rotina add128 (multiword.s) implementa a soma de dois numeros de 128 bits, cada um representado por um par de registradores de 64 bits. A propagacao do carry entre as duas metades e realizada atraves da instrucao adds (que atualiza a flag de carry do registrador de estado do processador) seguida de adc (que soma incluindo o carry propagado), reproduzindo o mecanismo de hardware utilizado em aritmetica de precisao estendida.

3.2 Conversao entre inteiro e ponto flutuante

As rotinas int_to_float e float_to_int (fpconv.s) utilizam, respectivamente, as instrucoes scvtf (signed convert to float) e fcvtzs (float convert to zero, signed), esta ultima truncando o resultado em direcao a zero, no mesmo comportamento de uma conversao explicita de tipo em linguagem C.

```

jotape@DARKIN:/mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tpi/assembly$ make clean
rm -rf build
jotape@DARKIN:/mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tpi/assembly$ make all
mkdir -p build
aarch64-linux-gnu-as -g -o build/multiword.o src/multiword.s
aarch64-linux-gnu-as -g -o build/fpconv.o src/fpconv.s
aarch64-linux-gnu-as -g -o build/lookup.o src/lookup.s
aarch64-linux-gnu-as -g -o build/bitops.o src/bitops.s
aarch64-linux-gnu-as -g -o build/main.o src/main.s
aarch64-linux-gnu-ld -o build/main build/multiword.o build/fpconv.o build/lookup.o build/bitops.o build/main.o
jotape@DARKIN:/mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tpi/assembly$ make run
qemu-aarch64 build/main
TPU routines OK
Exit code: 0
jotape@DARKIN:/mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tpi/assembly$ make disasm
aarch64-linux-gnu-objdump -d build/main > build/main_disasm.txt
Disassembly salvo em build/main_disasm.txt
jotape@DARKIN:/mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tpi/assembly$ make debug
Rode em outro terminal: gdb-multiarch build/main -> target remote localhost:1234
qemu-aarch64 -g 1234 build/main

jotape@DARKIN:/mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tpi/assembly$ gdb-multiarch
GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1
Copyright (C) 2015 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "aarch64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from build/main...
X /mnt/d/estudos/infnet/bloco8/pb/vga-framebuffer-arm-fpga/tpi/assembly/target: No such file or
directory.
(gdb) break float_to_int
Breakpoint 1 at 0x42280000: file src/fpconv.s, line 10.
(gdb) continue
X The program is not being run.
(gdb) target remote localhost:1234
Remote debugging using localhost:1234

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) n
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
_start () at src/main.s:9
9      mov     x0, #1
(gdb) continue
Continuing.

Breakpoint 1, float_to_int () at src/fpconv.s:10
10      fcvtzs  x0, s0
(gdb) p/x $s0
$1 = {f = 0x42280000, u = 0x42280000, s = 0x42280000}
(gdb) info registers x0
x0                0x2a                42
(gdb) next
11
(gdb) p $x0
$2 = 42
(gdb) |

```

Figura 6 - Sessão de depuração via GDB remoto sobre QEMU, com breakpoint na rotina `float_to_int`, evidenciando o valor de ponto flutuante em `s0` (0x42280000, representação IEEE-754 de 42,0) e o resultado inteiro convertido em `x0` (42).

3.3 Tabela de consulta (lookup table)

A rotina `lookup_square` (`lookup.s`) implementa uma tabela de consulta de valores pre-calculados (quadrados de 0 a 15), acessada por índice através de endereçamento indexado, evitando o recálculo em tempo de execução.

3.4 Mascaras, shifts e rotações

As rotinas `extract_field` e `rotate_left` (`bitops.s`) implementam manipulação bitwise avançada: a primeira extrai um campo de bits de um registrador através de deslocamento e mascaramento; a segunda realiza rotação lógica a esquerda, implementada como uma rotação a direita (ror, única instrução nativa de rotação no conjunto AArch64) de 32 menos o número de bits desejado.

3.5 Requisitos de avaliacao atendidos nesta etapa

Conversao entre inteiro e ponto flutuante: atendido, ver Figura 6 e Figura 7 (x22).

Mascaras, shifts, rotacoes e combinacoes bitwise: atendido, ver Figura 7 (x24/x25).

Restricao de ambiente: pela ausencia do Raspberry Pi Zero 2W fisico, a validacao das rotinas NEON foi realizada via QEMU, que emula corretamente o conjunto de instrucoes SIMD do AArch64. O ganho de desempenho por paralelizacao demonstrado nesta secao e estrutural (reducao do numero de instrucoes necessarias), e nao um beneficio de tempo de execucao medido, ja que a emulacao por software nao reflete de forma representativa o desempenho de hardware real.

Foram implementadas duas rotinas vetoriais utilizando os registradores NEON de 128 bits (v0-v31), tratados como vetores de quatro elementos de 32 bits (notacao .4s): `vector_add_int` (`neon_int.s`), que soma quatro pares de inteiros de 32 bits em uma unica instrucao ADD, e `vector_mul_float` (`neon_float.s`), que multiplica quatro pares de valores de ponto flutuante em uma unica instrucao FMUL. A versao escalar equivalente exigiria quatro instrucoes separadas para cada operacao, demonstrando o ganho estrutural de paralelizacao do SIMD.


```

jotape@DARKIN: /mnt/d/estudos/infnet/bloco8/ph/vga-framebuffer-asm-fpga/tp4/assembly$ make clean
rm -rf build
jotape@DARKIN: /mnt/d/estudos/infnet/bloco8/ph/vga-framebuffer-asm-fpga/tp4/assembly$ make all
mkdir -p build
aarch64-linux-gnu-as -g -o build/multiword.o src/multiword.s
aarch64-linux-gnu-as -g -o build/fpconv.o src/fpconv.s
aarch64-linux-gnu-as -g -o build/lookup.o src/lookup.s
aarch64-linux-gnu-as -g -o build/bitops.o src/bitops.s
aarch64-linux-gnu-as -g -o build/neon_int.o src/neon_int.s
aarch64-linux-gnu-as -g -o build/neon_float.o src/neon_float.s
aarch64-linux-gnu-as -g -o build/main.o src/main.s
aarch64-linux-gnu-ld -o build/main build/multiword.o build/fpconv.o build/lookup.o build/bitops.o build/neon_int.o build/neon_float.o build/main.o
jotape@DARKIN: /mnt/d/estudos/infnet/bloco8/ph/vga-framebuffer-asm-fpga/tp4/assembly$ make run
qemu-aarch64 build/main
TP4 rotinas OK
Exit code: 0
jotape@DARKIN: /mnt/d/estudos/infnet/bloco8/ph/vga-framebuffer-asm-fpga/tp4/assembly$ make debug
Rode em outro terminal: gdb-multiarch build/main -> target remote localhost:1234
qemu-aarch64 -g 1234 build/main

jotape@DARKIN: /mnt/d/estudo... $ gdb-multiarch
GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1
Copyright (C) 2015 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "aarch64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from build/main...
(gdb) target remote localhost:1234
Remote debugging using localhost:1234

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfed.ubuntu.com>
Enable debuginfo for this session? (y or [n]) n
Debuginfo has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
_start () at src/main.s:7
7      mov    x0, #2-1
(gdb) break vector_add_int
Breakpoint 1 at build/main: file src/neon_int.s, line 11.
(gdb) continue
Continuing.

Breakpoint 1, vector_add_int () at src/neon_int.s:11
11      adr    x0, vec_a_int
(gdb) finish
Run till exit from #0 vector_add_int () at src/neon_int.s:11
_start () at src/main.s:30
30      bl     vector_mul_float
(gdb) x/4d $vec_result_int
0x00000000: 11 22 33 44
(gdb) break vector_mul_float
Breakpoint 2 at build/main: file src/neon_float.s, line 11.
(gdb) continue
Continuing.

Breakpoint 2, vector_mul_float () at src/neon_float.s:11
11      adr    x0, vec_a_float
(gdb) finish
Run till exit from #0 vector_mul_float () at src/neon_float.s:11
_start () at src/main.s:30
30      adr    x1, #0
(gdb) x/4d $vec_result_float
0x00000000: 3 5 7 9
(gdb) |

```

Figura 8 - Execução das rotinas NEON via GDB remoto sobre QEMU: soma vetorial de inteiros (resultado 11, 22, 33, 44) e multiplicação vetorial de ponto flutuante (resultado 3, 5, 7, 9), lidos diretamente da memória após a execução das instruções ST1.

4.1 Requisitos de avaliação atendidos nesta etapa

Identificação de registradores e modos de operação NEON: atendido, registradores v0-v2 utilizados e documentados no código comentado.

Rotina vetorial para inteiros: atendido (vector_add_int), ver Figura 8.

Rotina vetorial para ponto flutuante: atendido (vector_mul_float), ver Figura 8.

Demonstração de ganho estrutural por paralelização: atendido via análise comparativa (1 instrução SIMD versus 4 instruções escalares), documentada nos comentários do código-fonte.

5 ATUALIZAÇÃO DA ARQUITETURA DO SISTEMA

A arquitetura do TP4 amplia a base estabelecida no TP3, mantendo a mesma separação entre validação física (Tang Nano 4K) e validação simulada via QEMU (Raspberry Pi Zero 2W, ainda indisponível fisicamente).

No lado FPGA, além da máquina de estados fsm_control já validada no TP3, foi adicionado o módulo mac_unit, um multiplicador-acumulador com saturação de 32 bits, que utiliza dois blocos BRAM (coeff_rom/sample_rom, 64 posições de 16 bits cada, com leitura síncrona registrada) para armazenar tabelas de coeficientes e amostras, e um bloco DSP (MULTADDALU18X18, confirmado no relatório de síntese do Gowin) para realizar a

multiplicacao em hardware dedicado. Um controlador de tres estados (`mac_control`) orquestra o acesso sequencial a BRAM, respeitando a latencia de leitura sincrona antes de disparar a operacao aritmetica.

No lado ARM, alem das rotinas de controle ja desenvolvidas no TP3, foram implementadas rotinas numericas avancadas (aritmetica multi-palavra de 128 bits, conversao entre inteiro e ponto flutuante, tabela de consulta e manipulacao bitwise avancada) e duas rotinas SIMD utilizando o conjunto de instrucoes NEON, cada uma processando quatro elementos em uma unica instrucao. Foi tambem implementado um mecanismo de comunicacao simulada entre ARM e FPGA (modulo `link.s`), com deteccao de erros via checksum aditivo e registro circular de telemetria.

O fluxo numerico ARM-FPGA e FPGA-ARM, que permitiria ao Raspberry Pi enviar dados para serem processados pela unidade MAC da FPGA e receber de volta o resultado acumulado, esta planejado na arquitetura mas ainda nao foi implementado fisicamente, dependendo da disponibilidade futura do hardware Raspberry Pi para validacao completa. A aproximacao com o protótipo final se da pela introducao de processamento numerico real (aritmetica saturada, ponto flutuante, paralelizacao SIMD e deteccao de erros de comunicacao) nos dois lados do sistema, blocos fundamentais que sustentarao o pipeline de processamento em tempo real do sistema integrado no TP5.

6 COMUNICACAO ARM-FPGA COM DETECAO DE ERRO

Restricao de ambiente: pela ausencia do Raspberry Pi Zero 2W fisico, o link de comunicacao ARM-FPGA foi simulado como uma regio de memoria de usuario (`link_buffer`), seguindo a mesma tecnica ja empregada no mapeamento GPIO do TP3. O objetivo desta etapa e demonstrar a logica de deteccao de erros e de registro de telemetria que sera necessaria quando a comunicacao fisica for implementada.

Foi implementado um checksum aditivo (`compute_checksum`), que soma os quatro bytes do payload de 32 bits modulo 256. A rotina `send_packet` monta o pacote na regio simulada de memoria, calculando e anexando o checksum. A rotina `receive_and_validate` le o pacote, recalcula o checksum sobre o payload recebido e compara com o checksum armazenado, retornando um indicador de sucesso ou falha. A rotina `log_telemetry` registra cada transacao (status e payload) em um buffer circular de quatro entradas, atendendo ao requisito de telemetria.

Dois cenarios foram testados: uma transmissao valida, em que o checksum recalculado confere com o armazenado, e uma transmissao deliberadamente corrompida (o byte de checksum e invertido via operacao XOR apos o envio), em que a falha e

corretamente detectada.

```

jotape@DARKIN:/mnt/d/estudos/infnet/blocos/pb/vga-framebuffer-azw-fpga/tpi/assembly$ make clean
rm -rf build
jotape@DARKIN:/mnt/d/estudos/infnet/blocos/pb/vga-framebuffer-azw-fpga/tpi/assembly$ make all
mkdir -p build
aarch64-linux-gnu-as -g -o build/multimord.o src/multimord.s
aarch64-linux-gnu-as -g -o build/fpconv.o src/fpconv.s
aarch64-linux-gnu-as -g -o build/lookup.o src/lookup.s
aarch64-linux-gnu-as -g -o build/bitops.o src/bitops.s
aarch64-linux-gnu-as -g -o build/neon_int.o src/neon_int.s
aarch64-linux-gnu-as -g -o build/neon_float.o src/neon_float.s
aarch64-linux-gnu-as -g -o build/link.o src/link.s
aarch64-linux-gnu-as -g -o build/main.o src/main.s
aarch64-linux-gnu-ld -o build/main build/multimord.o build/fpconv.o build/lookup.o build/bitops.o
build/neon_int.o build/neon_float.o build/link.o build/main.o
jotape@DARKIN:/mnt/d/estudos/infnet/blocos/pb/vga-framebuffer-azw-fpga/tpi/assembly$ make run
qemu-aarch64 build/main
TPU rotinas OK
Exit code: 0
jotape@DARKIN:/mnt/d/estudos/infnet/blocos/pb/vga-framebuffer-azw-fpga/tpi/assembly$ make debug
Rode em outro terminal: gdb-multiarch build/main -> target remote localhost:1234
qemu-aarch64 -g 1234 build/main
TPU rotinas OK
jotape@DARKIN:/mnt/d/estudos/infnet/blocos/pb/vga-framebuffer-azw-fpga/tpi/assembly$

ain
GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv2+: GNU GPL version 2 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from build/main...
(gdb) target remote localhost:1234
Remote debugging using localhost:1234

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) n
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
_start () at src/main.s:7
7
(gdb) break receive_and_validate
Breakpoint 1 at 0x00000000: file src/link.s, line 46.
(gdb) continue
Continuing.

Breakpoint 1, receive_and_validate () at src/link.s:46
46      stp    x18, x2r, [sp, #16]
(gdb) finish
Run till exit from #0 receive_and_validate () at src/link.s:46
_start () at src/main.s:103
103     mov    x26, x0
(gdb) p $x0
$1 = 0
(gdb) continue
Continuing.

Breakpoint 1, receive_and_validate () at src/link.s:46
46      stp    x18, x2r, [sp, #16]
(gdb) finish
Run till exit from #0 receive_and_validate () at src/link.s:46
_start () at src/main.s:103
103     mov    x27, x0
(gdb) p $x0
$1 = 1
(gdb) continue
Continuing.
[Inferior 1 (process 1504) exited normally]
(gdb) x/2gx $log_buffer
0x0000000000000000  0x0000000000000000
(gdb) |

```

Figura 9 - Sessão de depuração via GDB, mostrando o resultado da função `receive_and_validate` para a transmissão válida (retorno 0, sucesso) e para a transmissão corrompida (retorno 1, erro detectado).

```

jotape@DARKIN:/mnt/d/estudos/infnet/blocos/pb/vga-framebuffer-azw-fpga/tpi/assembly$ make clean
rm -rf build
jotape@DARKIN:/mnt/d/estudos/infnet/blocos/pb/vga-framebuffer-azw-fpga/tpi/assembly$ make all
mkdir -p build
aarch64-linux-gnu-as -g -o build/multimord.o src/multimord.s
aarch64-linux-gnu-as -g -o build/fpconv.o src/fpconv.s
aarch64-linux-gnu-as -g -o build/lookup.o src/lookup.s
aarch64-linux-gnu-as -g -o build/bitops.o src/bitops.s
aarch64-linux-gnu-as -g -o build/neon_int.o src/neon_int.s
aarch64-linux-gnu-as -g -o build/neon_float.o src/neon_float.s
aarch64-linux-gnu-as -g -o build/link.o src/link.s
aarch64-linux-gnu-as -g -o build/main.o src/main.s
aarch64-linux-gnu-ld -o build/main build/multimord.o build/fpconv.o build/lookup.o build/bitops.o
build/neon_int.o build/neon_float.o build/link.o build/main.o
jotape@DARKIN:/mnt/d/estudos/infnet/blocos/pb/vga-framebuffer-azw-fpga/tpi/assembly$ make run
qemu-aarch64 build/main
TPU rotinas OK
Exit code: 0
jotape@DARKIN:/mnt/d/estudos/infnet/blocos/pb/vga-framebuffer-azw-fpga/tpi/assembly$ make debug
Rode em outro terminal: gdb-multiarch build/main -> target remote localhost:1234
qemu-aarch64 -g 1234 build/main
TPU rotinas OK
jotape@DARKIN:/mnt/d/estudos/infnet/blocos/pb/vga-framebuffer-azw-fpga/tpi/assembly$

jotape@DARKIN:/mnt/d/estudos/infnet/blocos/pb/vga-framebuffer-azw-fpga/tpi/assembly$ gdb-multiarch build/m
GNU gdb (Ubuntu 17.1-2ubuntu1) 17.1
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv2+: GNU GPL version 2 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
<http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from build/main...
(gdb) target remote localhost:1234
Remote debugging using localhost:1234

This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) n
Debuginfod has been disabled.
To make this setting permanent, add 'set debuginfod enabled off' to .gdbinit.
_start () at src/main.s:7
7
(gdb) break log_telemetry
Breakpoint 1 at 0x00000000: file src/link.s, line 66.
(gdb) continue
Continuing.

Breakpoint 1, log_telemetry () at src/link.s:66
66      adr    x2, log_index
(gdb) finish
Run till exit from #0 log_telemetry () at src/link.s:66
_start () at src/main.s:106
106     adr    x1, link_buffer
(gdb) continue
Continuing.

Breakpoint 1, log_telemetry () at src/link.s:66
66      adr    x2, log_index
(gdb) finish
Run till exit from #0 log_telemetry () at src/link.s:66
_start () at src/main.s:106
106     mov    x1, msg
(gdb) x/2gx $log_buffer
0x0000000000000000  0x0000000000000001
(gdb) |

```

Figura 10 - Conteúdo do buffer de telemetria (`log_buffer`) após as duas transações, confirmando o registro correto do status (0 = sucesso, 1 = erro) e do payload (12345, 0x3039) de cada transação.

Durante o desenvolvimento desta etapa, dois defeitos de programação foram identificados e corrigidos, ambos relacionados a convenção de chamada AAPCS64: o primeiro consistia na ausência de preservação do registrador de retorno (x30) antes de uma chamada aninhada dentro de `receive_and_validate`, causando um laço infinito de retorno; o segundo consistia no uso de um registrador temporário (w3) que era

sobrescrito pela sub-rotina `compute_checksum` antes de ser utilizado na comparacao, levando a deteccao erronea de falha em transmissoes validas. Ambos os casos ilustram a importancia da disciplina de preservacao de registradores caller-saved e callee-saved em rotinas Assembly com chamadas aninhadas.

6.1 Requisitos de avaliacao atendidos nesta etapa

Comunicacao ARM-FPGA com envio e recebimento de dados numericos: atendido de forma simulada via QEMU, ver Figuras 9 e 10.

Deteccao de erros (checksum): atendido (`compute_checksum`, `receive_and_validate`), ver Figura 9.

Registro de telemetria (logs, status): atendido (`log_telemetry`, `log_buffer`), ver Figura 10.

7 CONSIDERACOES FINAIS

7.1 Sintese do status dos requisitos de avaliacao

- 1 - Arquitetura ARM-FPGA atualizada com BRAM, DSP, fluxos numericos e rotinas SIMD: atendido (Secao 5).
- 2 - Modulos aritmeticos em Verilog com BRAM, DSP, waveforms e teste fisico: atendido (Secao 2).
- 3 - Rotinas Assembly ARM64 com multi-palavra, conversao, tabelas e bitwise: atendido (Secao 3).
- 4 - Integracao ARM, FPGA e perifericos com ajustes eletricos e de sincronizacao: atendido do lado FPGA (Tang Nano 4K fisica, Secao 2.5); lado ARM permanece simulado via QEMU pela ausencia do Raspberry Pi fisico.
- 5 - Comunicacao ARM-FPGA com deteccao de erro e telemetria: atendido de forma simulada (Secao 6).
- 6 - Validacao da integracao parcial com documentacao intermediaria: atendido ao longo de todo o processo de desenvolvimento, incluindo o registro dos defeitos identificados e corrigidos (Secao 6).
- 7 - Medicao e documentacao de desempenho (ciclos, latencia, estabilidade): nao realizado nesta entrega; fica como pendencia para as proximas etapas.

8 - Integracao completa dos modulos com relatorio e video demonstrativo: relatorio tecnico atendido nesta entrega; video demonstrativo nao incluido, conforme decisao do autor.

7.2 Pendencias e proximos passos

Ficam pendentes para as proximas entregas: a medicao formal de desempenho (ciclos de clock, latencia das operacoes aritmeticas e estabilidade com sequencias de teste continuas), atualmente nao coberta por esta entrega; e a implementacao efetiva da comunicacao ARM-FPGA fisica, que permanece como trabalho planejado na arquitetura, dependente da disponibilidade futura do hardware Raspberry Pi Zero 2W.